Risk-Aware Payment Delegation: From Intent to Executable Decisions

Anonymous ACL submission

Abstract

Payment agents may need to commit to a purchase while relevant uncertainty remains and without further user interaction. Their decisions must respect interacting requirements and financial consequences while preserving opportunities for useful completion. We study risk-aware payment delegation as a language-to-decision problem: identify one purchase that is admissible across retained interpretations or contractual events and has positive worst-case value, or refuse when none exists. We present a state-wise intent DSL and an executable framework that carry decision-relevant alternatives from natural-language requests into hard requirements, risk conditions, and utility. A language-model front end instantiates this predefined representation; a fixed evaluator checks candidates and performs selection or abstention. Reference-checked contractual boundary tasks show improvements in both safe completion and justified refusal over direct answering. Cross-model, reasoning-effort, language, and reuse studies examine the scope and cost of these gains. Controlled intent-ambiguity tests and semantic diagnostics examine decisions across unresolved readings and errors that execution cannot repair. The formulation, implementation, and controlled testbed support the study of useful payment decisions within explicit risk boundaries.

1 Introduction

Language agents increasingly turn natural-language requests into actions through tools and external services (Yao et al., 2023). Shopping, travel-planning, and service benchmarks already test grounded choices, interacting requirements, and policy-governed workflows (Yao et al., 2022; Xie et al., 2024; Yao et al., 2025). Payment delegation adds a consequential decision: committing the user's resources to a purchase whose acceptability may depend on events that have not occurred. A purchase can meet its initial budget yet expose the user to an unacceptable cancellation or replacement cost. We study this setting with no further user interaction after the initial delegation. The agent must preserve the value of the delegation while respecting the requirements under which it may act.

Relevant uncertainty concerns both the meaning of a request and the consequences of a purchase. The request and available context may support several reasonable interpretations without specifying a unique intended reading; a contract may permit several future outcomes. Our setting excludes further user interaction and does not require either source of uncertainty to be resolved before acting. Computing the joint effect of conditional clauses adds a distinct difficulty. Selecting one interpretation or event prematurely can overlook an unacceptable consequence, while refusing solely because uncertainty remains can discard a purchase acceptable under every retained possibility. The central question is therefore: *can the agent identify one useful purchase that remains admissible across the retained interpretations and outcomes?*

Existing research provides complementary foundations. Uncertainty-aware planning and clarification methods determine when information should be requested (Ren et al., 2023; Zhang and Choi, 2025; Suri et al., 2026). Value of Information explicitly balances the benefit of asking against interaction costs, including in flight-booking and e-commerce tasks (Dong et al., 2026). Risk-Averse Calibration (RAC) relates uncertainty sets to consequence-sensitive action selection (Kiyani et al., 2025), while program-aided and neurosymbolic reasoning separate language interpretation from computation (Gao et al., 2023; Pan et al., 2023; Kesseli et al., 2025). These advances motivate a specific language-to-decision task: with no further user response, construct executable purchasing conditions from the request's requirements,

conditional scope, and financial consequences, then determine whether those conditions permit a useful action.

Our approach retains decision-relevant possibilities and expresses what the *same* purchase must satisfy in each. Hard requirements and authorized risk limits determine admissibility; utility ranks the admissible purchases. A favorable utility cannot compensate for a violated permission. Conversely, unresolved uncertainty is not itself a reason to refuse: an admissible purchase with positive guaranteed value remains actionable.

We present a *state-wise executable framework* for risk-aware payment delegation under unresolved uncertainty. Its predefined representation associates decision-relevant interpretations and outcomes with state-dependent requirements, risk conditions, and utility. This carries unresolved ambiguity into purchasing: one action must remain admissible across the retained possibilities, while its value is assessed by worst-case utility. Building on RAC's fixed-set max–min principle, a fixed procedure selects a positive-value maximizing purchase or abstains when none exists. Our implementation uses a language-model front end to instantiate the representation from the request, context, and attribute schema, followed by deterministic evaluation over candidate offers.

We evaluate this approach on 200 reference-checked tasks spanning 20 contractual structures with explicit event uncertainty. Across two DeepSeek configurations, exact decisions increase from 62.0% to 92.5%, while unsafe acceptances and false refusals both decrease. Separate controlled tests examine retained pronoun and scope ambiguity; reference interventions trace failures to omitted interpretations. Cross-model, effort, language, and reuse studies examine reliability and cost. These complementary studies support useful decisions within explicit risk boundaries. Our contributions are:

- **Risk-aware payment delegation:** a language-to-decision formulation that retains decision-relevant alternatives and separates admissibility, guaranteed value, and justified refusal without further user interaction.
- **An executable payment-decision framework:** an inspectable state-wise interface linking retained alternatives to requirements, financial permissions, and utility, with fixed decision semantics and reusable catalogue-independent specifications.
- **Controlled reliability evaluation:** reference-checked paired payment boundaries, cross-configuration studies, and semantic diagnostics that measure safe completion, missed feasible purchases, and remaining compilation failures.

Code, benchmark materials, retained model responses, evaluation records, and reproduction tools are publicly available in our anonymous artifact package (Appendix B.6).[1]

2 Related Work

Uncertainty and consequential decisions. KnowNo, IntentSim, and AmbiK examine uncertainty and clarification in planning (Ren et al., 2023; Zhang and Choi, 2025; Ivanova et al., 2025). VoI weighs the utility of information against user effort, while SAGE structures uncertainty over tool parameters (Dong et al., 2026; Suri et al., 2026). Related studies address abstention, ambiguous questions, and formal uncertainty (Wang et al., 2025; Liu et al., 2025; Gourabathina et al., 2026; Ganguly et al., 2025). RAC connects prediction sets to risk-averse decisions; risk-control and action-conditional extensions add statistical guarantees under their assumptions (Kiyani et al., 2025; Angelopoulos et al., 2024; Farinhas et al., 2024; Zhu et al., 2026; Quach et al., 2024). Daheim and Gurevych (2026) apply Bayesian and risk-averse decisions to tutoring and reviewing. We use the inherited decision principle for executable purchases with explicit financial permissions, a common decision across states, and an outside option. The operating condition excludes further clarification; it does not imply that prior interaction methods cannot act under uncertainty.

Executable reasoning and formalization. PAL, Logic-LM, LINC, and Logic.py delegate computation to interpreters, provers, or solvers (Gao et al., 2023; Pan et al., 2023; Olausson et al., 2023; Kesseli et al., 2025). Formalization studies investigate the fidelity of language-to-logic translation and its sensitivity to natural descriptions (Liu, 2025; Pei et al., 2025; Huang and Zhang, 2025; Quansah et al., 2026). We inherit the separation of compilation and execution, with state-dependent purchas-

[1] https://anonymous.4open.science/r/risk-aware-payment-delegation-D4FC/

ing conditions as the generated object. Separating stable requirements from catalogue data also limits the need to repeatedly process large contexts (Liu et al., 2024). Our evaluation measures the entire decision architecture; supplementary execution controls distinguish shared computational benefits from representation-dependent failures.

Constrained agents and evaluation. GuardAgent generates executable safeguards, DeAction corrects proposed actions, AgentSpec enforces structured runtime constraints, and VeriGuard verifies policies and actions (Xiang et al., 2025; Ning et al., 2026; Wang et al., 2026; Miculicich et al., 2025). AutoCO studies executable constraint-relaxation strategies; financial metric graphs constrain derivations (Liu et al., 2026; Jiang et al., 2026). Our focus is the completion/refusal boundary of a purchase under retained states, including separate admissibility and value conditions. PlanBench, CoPE, FollowBench, ComplexBench, and AgentIF test planning or composed instructions (Valmeekam et al., 2023; Huang et al., 2026; Jiang et al., 2024; Wen et al., 2024; Qi et al., 2025); ToolEmu, AgentDojo, and AgentHarm assess consequential or adversarial agent behavior (Ruan et al., 2024; Debenedetti et al., 2024; Andriushchenko et al., 2025). We contribute controlled contractual boundaries and semantic diagnostics, drawing on behavioral testing and contrastive evaluation (Ribeiro et al., 2020; McCoy et al., 2019; Gardner et al., 2020). Authorization-domain auditing supplements decision evaluation within this task.

3 Risk-Aware Payment Delegation: Formulation and Execution

The framework carries decision-relevant alternatives from a natural-language delegation into a state-wise representation, then applies predefined risk and value rules to decide whether and how to act.

3.1 Decision Setting and Requirements

Let $x = (P, c)$ contain a natural-language request P and context c, and let $z \in \mathcal{Z}$ describe a complete candidate purchase. A nonempty $\Theta(x)$ contains retained interpretations and/or relevant future events. States index conditions under which the same purchase is evaluated; a unique intended interpretation need not be identifiable. No further user interaction is available in this task. The catalogue $C \subseteq \mathcal{Z}$ is finite; state probabilities need not be available.

For each state, $H_\theta(x, z)$ encodes hard requirements, $\ell_\theta(x, z)$ financial consequences, $\mathcal{A}_\theta(x)$ the authorized risk region, and $u_\theta(x, z)$ utility on a common cardinal scale. For decision analysis, we assume that the representation preserves the request's decision-relevant interpretations, events, and associated requirements, consequences, risk permissions, and utility. Define

$$F_\theta(x) = \{z : H_\theta(x, z) = 1,\ \ell_\theta(x, z) \in \mathcal{A}_\theta(x)\}, \tag{1}$$

$$F(x) = \bigcap_{\theta \in \Theta(x)} F_\theta(x), \tag{2}$$

$$\psi_x(z) = \inf_{\theta \in \Theta(x)} u_\theta(x, z). \tag{3}$$

Equation (1) checks one state's requirements and consequences. Equation (2) requires one purchase to pass every such check: $\exists z\, \forall \theta$, not a different purchase after observing each state. Equation (3) gives that purchase's lowest utility over the same states. They jointly define

$$\max_{z \in C \cap F(x)} \psi_x(z), \tag{4}$$

with abstention worth zero. An empty feasible domain or a nonpositive maximum requires abstention; otherwise a maximizer should be selected. Admissibility and value are separate: a high score cannot authorize an outcome outside $F(x)$.

The max–min principle for a fixed uncertainty set follows RAC (Kiyani et al., 2025). Here, explicit purchasing permissions define feasibility; we implement the decision layer without conformal calibration. Context fixes risk permissions and utility before optimization. An allowance for extra spending, when specified, changes the relevant predicate rather than being inferred from infeasibility. The formulation also accommodates retained linguistic interpretations (Appendix A.1); the primary evaluation supplies finite contractual event families and stipulated utilities.

3.2 State-Wise Intent DSL

We define a lightweight domain-specific language (intent DSL), $\mathcal{L}$, as the executable intermediate representation between a natural-language delegation and the decision procedure. Executable request languages connect semantic parsing to task execution

(Lam et al., 2022); financial contract languages associate domain constructs with explicit interpretation and valuation rules (Peyton Jones et al., 2000). Our representation carries retained interpretations and contractual outcomes into risk-aware purchase selection.

An intent specification $Q \in \mathcal{L}$ contains a nonempty collection of state-indexed clauses, abstractly

$$Q = \left\{(\widehat{H}_j, \widehat{R}_j, \widehat{u}_j)\right\}_{j=1}^{m}.$$

The predicates $\widehat{H}_j$ and $\widehat{R}_j$ encode hard requirements and admissibility of financial consequences under the same state; $\widehat{u}_j$ encodes its utility. Expressions use attributes from schema Σ with fixed Boolean and numeric semantics. A risk predicate realizes the test $\ell_\theta(x, z) \in \mathcal{A}_\theta(x)$ in Equation (1). Conditional scope, compensation order, and utility binding belong to these expressions. A loss cap bounds an admissible consequence; it is not compensation that reduces that consequence.

Clauses retain alternative readings or circumstances under which the same purchase is evaluated. Joint admissibility does not assert that the user intended their conjunction. The evaluator fixes cross-state conjunction and minimum utility (Section 3.1); the DSL exposes their state-specific inputs for inspection. This separates generated decision conditions from the fixed selection procedure. Equivalent decompositions preserve joint admissibility and worst-case value.

A semantic front end instantiates $Q = f(x, \Sigma)$ from the request and context. Our implementation uses a language model for f; the language, evaluation semantics, and refusal rule remain predefined. Benchmark inputs specify event families, risk permissions, and utility definitions; the model encodes their consequences. Primary compilation excludes candidate catalogues, identifiers, and ordering. A specification can be reused when requirements, state semantics, and attribute meanings remain unchanged. Concrete syntax, serialization, and interpreter details appear in Appendix A.2; they do not define the scope of the framework.

3.3 Decision Procedure

Algorithm 1 exactly solves the compiled instance of Equation (4) on a complete finite catalogue once the evaluation checks below succeed. This guarantee concerns execution of Q; natural-language compilation errors remain part of the end-to-end evaluation. Hats distinguish compiled quantities from reference semantics: $\widehat{F}_C$ passes all generated hard and risk conditions, and $\widehat{\psi}$ is minimum generated utility. Before selection, every candidate's admissibility must be decided and every admissible candidate's state utilities known. The procedure returns all positive-value maximizers using exact integer/rational arithmetic, preserving inclusive bounds and one-subunit differences.

Algorithm 1 Compile once; evaluate risk and value

Input: request/context x, schema Σ, catalogue C
Output: ACCEPT, REFUSE, or FAIL, with diagnostics

1 $Q \leftarrow f(x, \Sigma)$ *model compilation; no C*
2 Decode and validate Q against the interface.
3 If invalid, return FAIL(compilation).
4 Evaluate every generated state on every $z \in C$; retain the condition/value matrix.
5 If execution is invalid, return FAIL(execution).
6 For each $z \in C$, set $\widehat{f}(z) = \bigwedge_j (\widehat{H}_j(z) \wedge \widehat{R}_j(z))$.
7 If catalogue coverage or any $\widehat{f}(z)$ is unresolved, or a candidate with $\widehat{f}(z) = 1$ has an unknown state utility, return FAIL(uncertified).
8 $\widehat{F}_C \leftarrow \{z \in C : \widehat{f}(z) = 1\}$.
9 If $\widehat{F}_C = \varnothing$, return REFUSE(infeasible).
10 For $z \in \widehat{F}_C$, set $\widehat{\psi}(z) = \min_j \widehat{u}_j(z)$.
11 $v \leftarrow \max_{z \in \widehat{F}_C} \widehat{\psi}(z)$.
12 If $v \leq 0$, return REFUSE(nonpositive value).
13 Return ACCEPT($\{z \in \widehat{F}_C : \widehat{\psi}(z) = v\}, v$).

Fixed evaluator: lines 2–13. Reference answers are used only in offline evaluation.

Unresolved evidence prevents certification of feasibility or optimality. A known feasible candidate cannot be certified optimal while another potentially feasible candidate has unknown value; known violations already exclude a candidate regardless of its utility. Primary inputs supply complete catalogues and known attributes, although undefined generated expressions can still fail evaluation. For archived executions with unresolved inputs, agreement on a certified subset does not establish Algorithm 1's complete-domain guarantee (Appendix A.2). For M candidates, K generated states, and expression-evaluation cost T, exhaustive execution takes $O(MKT)$ work.

3.4 Scope and Semantic Failure Modes

Execution faithfully applies Q; preserving the request requires the generated states, permissions, and utility to be faithful as well. Omitting an event, misbinding a clause, or subtracting a risk allowance as if it were compensation can change $\widehat{F}_C$. Miscomputing consequences can change either feasibility or ranking. These are semantic compilation errors even when the evaluator runs successfully.

Evaluation dimension	Controlled scope	Question answered
Decision boundaries	200 tasks; 57 events per candidate	Authorized acceptance and justified refusal
Structural coverage	20 templates; 4 families; 100 paired boundaries	Coupled clauses and tight financial thresholds
Model configurations	11 model configurations	Reliability across model settings
Reasoning effort	4,500 evaluations; 25 complete configurations	Dependence on reasoning settings
Task language	4 profiles; 1,440 archived + 800 updated outcomes	Transfer under preserved financial semantics
Authorization audit	648 DeepSeek artifacts; 1,296 replays	Admitted domains beyond selected outcomes
Component diagnosis	16 selected faulty contracts	Distinct permission and utility errors
Retained intent ambiguity	3 families; 544 recorded API evaluations	Common decisions across unresolved readings
Rule reuse	2 requirement families; 16 catalogue snapshots	Repeated decisions and amortized model use

Table 1: **Complementary evidence, with explicit units.** Cohorts overlap; their counts are not summed as independent samples. Study-specific protocols, outcome counts, and controls are detailed in the appendix.

We test them offline against finite references through three complementary checks. *Decision correctness* checks the selected maximizers and their guaranteed value. *Authorization-domain correctness* checks $\widehat{F}_C \subseteq C \cap F(x)$, including unselected candidates. *Refusal correctness* checks that no reference-admissible candidate has positive guaranteed value. A safe current selection can mask an unsafe admitted alternative, while an overly restrictive rule can cause a false refusal. These diagnostics characterize the method's failures; they are not an additional runtime oracle. Further denotation properties and assumptions appear in Appendix A.

The evaluation concerns executable meaning rather than identical expression strings. Equivalent state decompositions can produce the same admissibility and minimum utility. Conversely, syntactically valid expressions with plausible labels can encode a different decision problem. The finite task therefore makes semantic comparisons possible through candidate-level conditions and values, independently of the surface representation.

When requirements, state functions, permissions, and attribute meanings remain fixed, Q can be reused on another catalogue. Our reuse study evaluates this operational property, and frozen-rule replays examine whether a change in availability exposes a latent semantic error (Sections 5.4 and 5.5).

4 Experimental Setup

Controlled payment boundaries. We construct 200 tasks from 20 contract templates in four families: coupon cancellation, insured cancellation, threshold-dependent returns, and service replacement. Contracts combine overlapping groups, compensation layers, shipping charges, and item/order caps. Each purchase has six components. The event universe contains all affected subsets of size at most four, including the empty event: 57 states per candidate. Catalogues contain 128–1,024 candidates. All amounts use abstract monetary units (MU), with 100 integer subunits per MU. Exact arithmetic on subunits preserves strict and inclusive financial boundaries.

Each template contributes five feasible/infeasible pairs. A feasible task has exactly one authorized positive-value candidate. Changing only its administrative fee moves its worst loss one subunit above the limit; other attributes and offer order stay fixed. These 100 pairs test boundary adherence and solution finding (Ribeiro et al., 2020; Gardner et al., 2020; McCoy et al., 2019). Separate cash-ledger and expression implementations agree on 6,712,320 candidate–event checks, supplemented by 25,600 random-attribute checks and three hand-calculated witnesses. Both implementations arose within one development workflow. Inputs and references were frozen before primary calls (Appendix B).

Comparisons. The DIRECT–SET comparison evaluates the complete executable decision architecture. DIRECT receives the request, schema, full catalogue, and robust decision rules; it returns a decision, selected identifiers, and guaranteed utility. SET compiles a catalogue-independent rule for the fixed evaluator under the same decision obligations. Procedural execution controls, including a matched-input comparison, examine shared computational gains and realization-dependent outcomes (Appendix E.2, Table 16).

The primary comparison spans eleven model configurations: Astra max; Sol, Terra, and Luna medium; DeepSeek low and high; Qwen Flash and Max low; GLM Flash low and GLM high; and Opus 5.5 medium. Cohort definitions are in Appendix B.5; other panels retain their own cohorts.

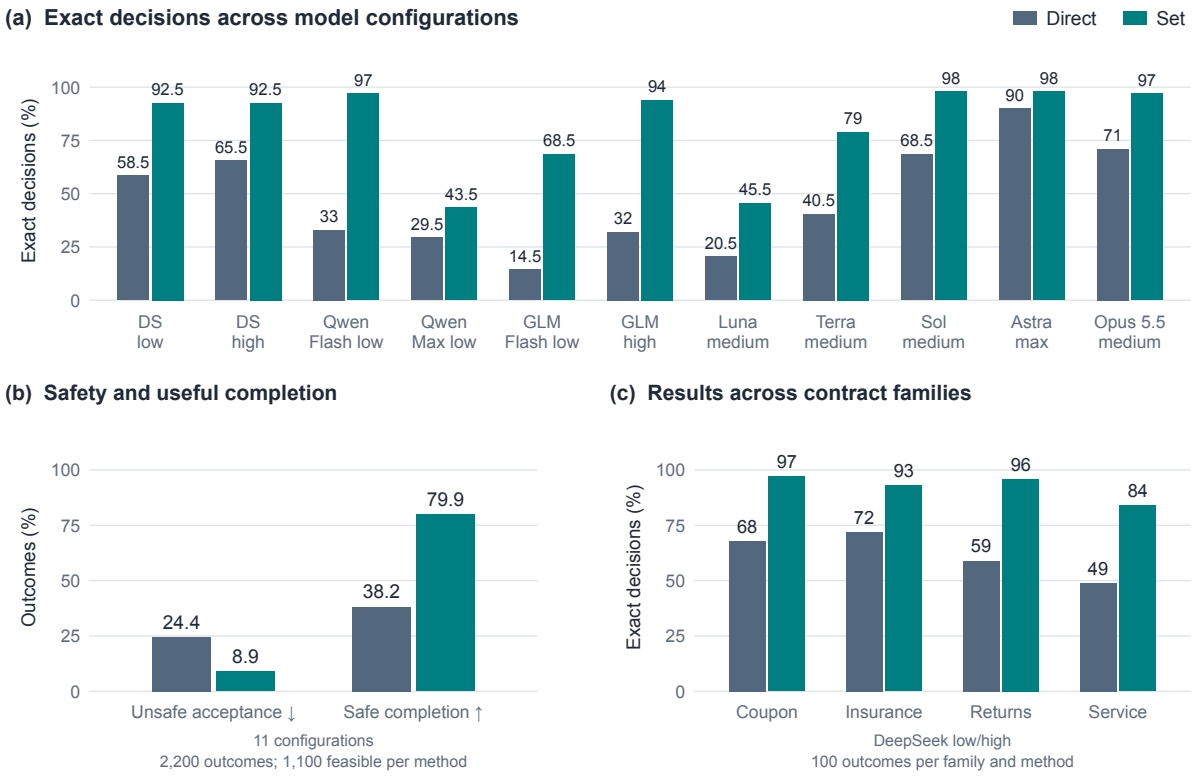


Figure 1: **Risk-aware decisions across models and contract structures.** Each group compares Direct and Set. (a) Exact-decision rates across model configurations. (b) All eleven configurations: unsafe acceptance over 2,200 task–configuration outcomes and safe completion over 1,100 feasible outcomes per method. (c) DeepSeek low/high across four contract families, each pooling 100 outcomes from five templates. All axes start at zero; failures remain in the denominators. Configurations share task sets.

Outcome scoring. An *exact decision* returns the reference decision, authorized maximizing choices, and correct utility. *Safe task completion* is exact acceptance on a feasible task. *Unsafe acceptance* returns a reference-infeasible choice; *false refusal* rejects an available positive-value authorized purchase. Other groups remaining value and execution failures. Execution failures are never credited as correct refusals; all failures remain in each completed cohort's denominator (Appendix B). Authorization auditing also checks unselected admitted candidates. Archived trials retain one semantic attempt, bounded transport retries, responses, usage, duration, and evaluation. Syntax recovery preserves expression meaning. Comparisons are paired within configurations; service-specific compute budgets are not equivalent.

Reproducibility. The anonymous artifact provides code, benchmark inputs and references, prompts, configurations, retained responses, and evaluation records. An offline entry reconstructs tables and the four main figures from frozen reporting inputs, checks paired outcome partitions and Algorithm 1, and records input hashes without model calls. Appendix B.6 details the material coverage and workflow; Table 7 specifies generation settings.

5 Results and Analysis

Table 1 maps nine evaluation dimensions: decision boundaries, structural coverage, model and effort settings, language, retained intent, authorization audits, semantic interventions, and reuse. The following results connect useful completion, robustness, and operational value; Appendices B and E give cohort definitions and controls.

5.1 Safe Completion under Contractual Uncertainty

Figure 1 combines model coverage, joint safety–completion outcomes, and contract-level variation. All eleven configurations improve exactness, reduce unsafe acceptances, and complete more authorized purchases. Across the two primary DeepSeek configurations, exact decisions rise from 248/400 (62.0%) to 370/400 (92.5%). The gain is 30.5 per-

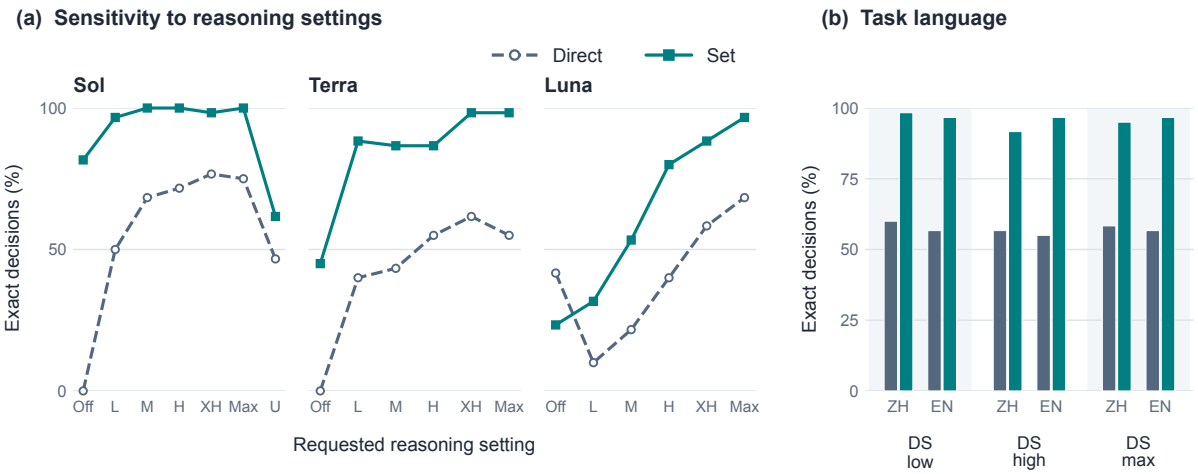


Figure 2: **Sensitivity to reasoning settings and task language.** (a) Direct and Set rates for Sol, Terra, and Luna across 19 complete settings on the 60-task effort panel. Lines connect requested settings, not measured compute. (b) Chinese (ZH) and English (EN) rates for DeepSeek low/high/max, with 60 tasks per language and method. All failures remain in the denominators. Cohorts overlap and are not pooled; additional counts and controls appear in the appendix.

centage points, with a 95% template-cluster bootstrap interval of $[22.75, 37.75]$, retaining both settings within each template. Exact purchases rise from 100 to 186 across 200 feasible outcomes, while unsafe acceptances fall from 79 to 9 and false refusals from 53 to 1. Safety improves together with useful completion.

Each paired task changes only one fee and requires the opposite decision. Joint metrics test responsiveness to financial conditions, beyond blanket acceptance or refusal.

The updated Astra max comparison scores 180/200 (90%) for direct answering and 196/200 (98%) for SET. Direct has six unsafe acceptances, six false refusals, and eight Other failures; SET has two unsafe acceptances and two false refusals. Traced errors from the original archived run illustrate both boundary failures (Appendix F.4).

5.2 Decisions under Unresolved User Intent

We audit 544 recorded API evaluations across three unresolved-intent families: ticket-holder reference, conditional hotel cancellation, and per-recipient versus whole-batch exchange-cost limits. Both methods receive the same catalogue, explicit ambiguity cues, and decision rules, without further user input. Figure 3 summarizes the complete paired API cohorts, separately from the primary event benchmark.

Exact-decision rates improve across the four displayed API configurations. The study spans risk limits, penalties, and the positive-value boundary; additional capability checks cover common purchases, empty cross-interpretation intersections, and max–min ranking (Appendix D).

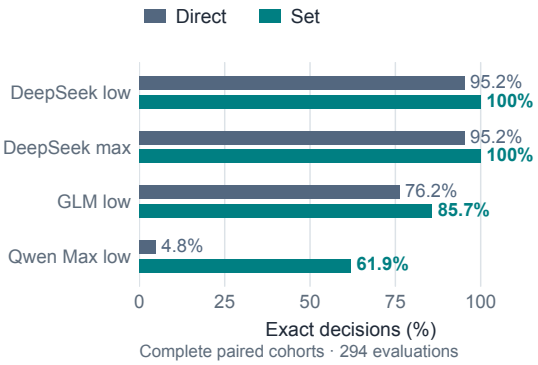


Figure 3: **Decisions with unresolved user intent.** Exact-decision rates in complete paired API cohorts. Percentages use each configuration's own denominator, retaining failures. Counts, error categories, and the full study are in Appendix D.

5.3 Robustness across Structures and Settings

The improvement occurs in all four financial families (Figure 1c) and in 18 of 20 templates at each DeepSeek setting (Appendix C). These are controlled structural checks, not repetitions of one numerical witness. The model overview in Figure 1a includes GLM high rising from 64/200 to 188/200

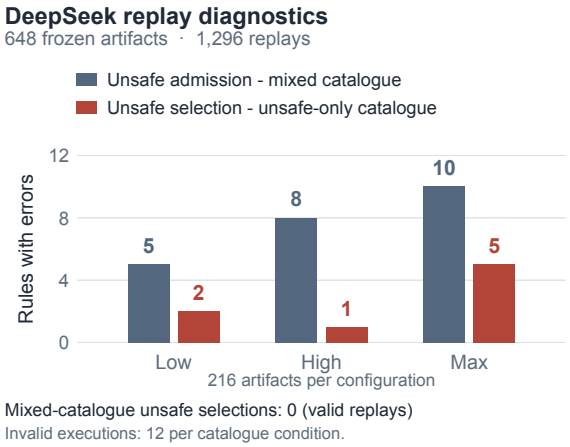


Figure 4: **Targeted DeepSeek replay diagnostics.** Bars compare unsafe admitted domains in mixed catalogues with unsafe selections in unsafe-only catalogues. Each configuration pools 216 frozen artifacts across three executable realizations. Invalid executions are recorded separately.

and Sol medium from 137/200 to 196/200. Sol's 21 unsafe acceptances and 42 false refusals under Direct become two of each under SET. Opus medium improves from 142/200 to 194/200, with exact purchases rising from 76 to 98 of 100 feasible tasks. Improvement is not uniform: Qwen Max low rises from 59 to 87 exact outcomes but still makes 46 unsafe acceptances, while Luna medium also retains errors. Execution and faithful compilation remain distinct requirements. Outcome partitions and cohort definitions are in Appendices C and B.5.

On the 60-task effort panel shown in Figure 2a, SET improves exactness in 18 of 19 complete GPT-5.6 configurations; Luna with reasoning disabled declines from 25/60 to 14/60. The curves also show nonmonotonic changes, including Sol's decline at ultra. These are end-to-end outcomes that retain compilation and operational failures. Language changes preserve identifiers, quantities, events, and authorization. The paired DeepSeek results in Figure 2b retain improvements in both languages. Full effort and language controls are collected in Appendix E. Budget controls recover some incomplete answers but leave semantic errors (Appendix E.5). These are controlled configuration and language tests, not unrestricted deployment generalization.

5.4 Targeted Diagnostics

We replay 648 frozen DeepSeek artifacts from 72 tasks and three effort settings, using Set and procedural controls with matched or visible catalogue input (Appendix F). Only catalogue availability changes. Across 1,296 replays, valid mixed-catalogue executions make no unsafe selections, although 23 rules admit an unauthorized offer. In an unsafe-only catalogue, eight select it (Figure 4); 12 artifacts fail execution per condition. Candidate availability can thus expose latent semantic errors. Reference-assisted state and component interventions isolate omitted or misrepresented conditions (Appendix F.2). These are offline diagnostics; improving semantic construction and validation remains future work.

Astra workflow	Calls	Tokens	Exact
Regenerate per catalogue	16	175,429	16/16
Reuse compiled rule	2	22,160	16/16

Table 2: **Reuse includes the first compilation.** Two fixed requirements, eight catalogues each; tokens include recorded input and output usage.

5.5 Reuse and Execution Cost

Across two fixed requirements and eight catalogue snapshots each, Astra uses two compilations and 22,160 tokens instead of 16 calls and 175,429 tokens, with 16/16 exact decisions in both routes (Table 2). The 87.4% reduction includes initial compilation. Reuse requires stable authorization; changed requirements trigger recompilation. Appendix E.6 reports the Direct/Set comparison and resource accounting.

6 Conclusion

We study payment delegation without further user interaction while uncertainty remains. An inspectable state-wise intent interface connects purchasing requirements, financial permissions, and utility to a fixed decision procedure. On the primary boundary benchmark, this executable framework improves authorization adherence and useful completion over direct answering. Retained-intent tests, semantic diagnostics, and cross-configuration studies examine these decisions; catalogue reuse reduces repeated model calls. Together, the formulation, implementation, and testbed support useful payment decisions within explicit risk boundaries.

Limitations

Semantic and decision scope. The experiments use synthetic purchasing contracts, complete catalogues, correct item grounding, known attributes, and finite uncertainty sets. The primary benchmark concerns contractual events; the historical intent panel supplies explicit cues to unresolved pronouns and scope. Open-ended discovery of plausible interpretations remains to be evaluated. Preferences and risk boundaries are specified rather than learned. Faithfully instantiating ambiguity and contractual semantics remains a limitation of the current front end. End-to-end evaluations retain representation errors. Conformal coverage would require additional calibration. Actual settlement, hidden merchant terms, fraud, and incomplete inventory are outside the tested task.

Evaluation and implementation scope. The primary benchmark deliberately pairs numerical boundaries with one feasible candidate or none. It supports controlled boundary comparisons, not deployment error-rate estimates or broad ranking over many feasible purchases. The references are separate implementations within one development workflow. Historical diagnostics, effort settings, translations, and replays share cases and are reported as distinct cohorts. Catalogue visibility and service compute budgets vary by architecture or provider; matched-input and execution controls are retained in the appendix. Some generated specifications remain incorrect or invalid. Primary effects concern the full compilation-and-execution pipeline; Appendix E.2 examines realization-dependent outcomes. Operational failures remain in their designated denominators.

Ethical Considerations

All purchases are synthetic, with no monetary transfers or customer records. Financial consequences follow stipulated benchmark contracts. Saved artifacts distinguish original responses, reference-assisted interventions, and subsequent analysis.

References

Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Yarin Gal, and Xander Davies. 2025. Agentharm: A benchmark for measuring harmfulness of llm agents. In *International Conference on Learning Representations*, volume 2025, pages 79185–79220.

Anastasios Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. 2024. Conformal risk control. In *International Conference on Learning Representations*, volume 2024, pages 55198–55218.

Nico Daheim and Iryna Gurevych. 2026. Uncertainty-aware generation and decision-making under ambiguity. arXiv preprint arXiv:2606.30578.

Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *Advances in Neural Information Processing Systems*, volume 37, pages 82895–82920. Curran Associates, Inc.

Yijiang River Dong, Tiancheng Hu, Zheng Hui, Caiqi Zhang, Ivan Vulić, Andreea Bobu, and Nigel Collier. 2026. Value of information: A framework for human–agent communication. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 42879–42896, San Diego, California, United States. Association for Computational Linguistics.

Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392, Melbourne, Australia. Association for Computational Linguistics.

António Farinhas, Chrysoula Zerva, Dennis Ulmer, and Andre Martins. 2024. Non-exchangeable conformal risk control. In *International Conference on Learning Representations*, volume 2024, pages 50952–50966.

Debargha Ganguly, Vikash Singh, Sreehari Sankar, Biyao Zhang, Xuecen Zhang, Srinivasan Iyengar, Xiaotian Han, Amit Sharma, Shivkumar Kalyanaraman, and Vipin Chaudhary. 2025. Grammars of formal uncertainty: When to trust llms in automated reasoning tasks. In *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 34213–34255. Curran Associates, Inc.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10764–10799. PMLR.

Matt Gardner, Yoav Artzi, Victoria Basmov, Jonathan Berant, Ben Bogin, Sihao Chen, Pradeep Dasigi, Dheeru Dua, Yanai Elazar, Ananth Gottumukkala, Nitish Gupta, Hannaneh Hajishirzi, Gabriel Ilharco,

Daniel Khashabi, Kevin Lin, Jiangming Liu, Nelson F. Liu, Phoebe Mulcaire, Qiang Ning, and 7 others. 2020. Evaluating models' local decision boundaries via contrast sets. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1307–1323, Online. Association for Computational Linguistics.

Abinitha Gourabathina, Inkit Padhi, Manish Nagireddy, Subhajit Chaudhury, and Prasanna Sattigeri. 2026. Answering the wrong question: Reasoning trace inversion for abstention in LLMs. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13307–13324, San Diego, California, United States. Association for Computational Linguistics.

Cassie Huang, Stuti Mohan, Ziyi Yang, Stefanie Tellex, and Li Zhang. 2026. Language model as planner and formalizer under constraints. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13724–13756, San Diego, California, United States. Association for Computational Linguistics.

Cassie Huang and Li Zhang. 2025. On the limit of language models as planning formalizers. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4880–4904, Vienna, Austria. Association for Computational Linguistics.

Anastasia Ivanova, Bakaeva Eva, Zoya Volovikova, Alexey Kovalev, and Aleksandr Panov. 2025. AmbiK: Dataset of ambiguous tasks in kitchen environment. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 33216–33241, Vienna, Austria. Association for Computational Linguistics.

Aoyuan Jiang, Liang Hong, Haoxuan Liu, and Rui Wang. 2026. Achieving multi-hop calculation and safe abstention in financial numerical reasoning by metric graph constrained LLMs. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27575–27595, San Diego, California, United States. Association for Computational Linguistics.

Yuxin Jiang, Yufei Wang, Xingshan Zeng, Wanjun Zhong, Liangyou Li, Fei Mi, Lifeng Shang, Xin Jiang, Qun Liu, and Wei Wang. 2024. Follow-Bench: A multi-level fine-grained constraints following benchmark for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4667–4688, Bangkok, Thailand. Association for Computational Linguistics.

Pascal Kesseli, Peter O'Hearn, and Ricardo Cabral. 2025. Logic.py: Bridging the gap between llms and constraint solvers. In *Advances in Neural Information Processing Systems*, volume 38, Main Conference, pages 62505–62525. Curran Associates, Inc.

Shayan Kiyani, George J. Pappas, Aaron Roth, and Hamed Hassani. 2025. Decision theoretic foundations for conformal prediction: Optimal uncertainty quantification for risk-averse agents. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 30943–30965. PMLR.

Monica S. Lam, Giovanni Campagna, Mehrad Moradshahi, Sina J. Semnani, and Silei Xu. 2022. ThingTalk: An extensible, executable representation language for task-oriented dialogues. *arXiv preprint arXiv:2203.12751*.

Beidan Liu, Zhengqiu Zhu, Chen Gao, Tianle Pu, Yong Zhao, Wei Qi, and Quanjun Yin. 2026. Learn to relax with large language models: Solving constraint optimization problems via bidirectional coevolution. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1075–1094, San Diego, California, United States. Association for Computational Linguistics.

Jingyu Liu, Jingquan Peng, Xiaopeng Wu, Xubin Li, Tiezheng Ge, Bo Zheng, and Yong Liu. 2025. Do not abstain! identify and solve the uncertainty. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17177–17197, Vienna, Austria. Association for Computational Linguistics.

Junnan Liu. 2025. Few-shot natural language to first-order logic translation via code generation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 10939–10960, Albuquerque, New Mexico. Association for Computational Linguistics.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

R. Thomas McCoy, Ellie Pavlick, and Tal Linzen. 2019. Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3428–3448, Florence, Italy. Association for Computational Linguistics.

Lesly Miculicich, Mihir Parmar, Hamid Palangi, Krishnamurthy Dj Dvijotham, Mirko Montanari, Tomas Pfister, and Long T. Le. 2025. Veriguard: Enhancing llm agent safety via verified code generation. arXiv preprint arXiv:2510.05156.

Yuting Ning, Jaylen Jones, Zhehao Zhang, Chentao Ye, Weitong Ruan, Junyi Li, Rahul Gupta, and Huan Sun. 2026. When actions go off-task: Detecting and correcting misaligned actions in computer-use agents.

In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 93346–93375. PMLR.

Theo Olausson, Alex Gu, Ben Lipkin, Cedegao Zhang, Armando Solar-Lezama, Joshua Tenenbaum, and Roger Levy. 2023. LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5153–5176, Singapore. Association for Computational Linguistics.

Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. 2023. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 3806–3824, Singapore. Association for Computational Linguistics.

Yu Pei, Yongping Du, and Xingnan Jin. 2025. FoVer: First-order logic verification for natural language reasoning. *Transactions of the Association for Computational Linguistics*, 13:1340–1359.

Simon Peyton Jones, Jean-Marc Eber, and Julian Seward. 2000. Composing contracts: An adventure in financial engineering. In *Proceedings of the ACM SIGPLAN International Conference on Functional Programming*. ACM.

Yunjia Qi, Hao Peng, Xiaozhi Wang, Amy Xin, Youfeng Liu, Bin Xu, Lei Hou, and Juanzi Li. 2025. Agentif: Benchmarking large language models instruction following ability in agentic scenarios. In *Advances in Neural Information Processing Systems*, volume 38, Main Conference. Curran Associates, Inc.

Victor Quach, Adam Fisch, Tal Schuster, Adam Yala, Jae Ho Sohn, Tommi Jaakkola, and Regina Barzilay. 2024. Conformal language modeling. In *International Conference on Learning Representations*, volume 2024, pages 11654–11681.

Paapa Quansah, Pablo Rivas, and Ernest Bonnah. 2026. Verify: A novel multi-domain dataset grounding ltl in contextual natural language via provable intermediate logic. In *International Conference on Learning Representations*, volume 2026, pages 86416–86453.

Allen Z. Ren, Anushri Dixit, Alexandra Bodrova, Sumeet Singh, Stephen Tu, Noah Brown, Peng Xu, Leila Takayama, Fei Xia, Jake Varley, Zhenjia Xu, Dorsa Sadigh, Andy Zeng, and Anirudha Majumdar. 2023. Robots that ask for help: Uncertainty alignment for large language model planners. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 661–682. PMLR.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912, Online. Association for Computational Linguistics.

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris Maddison, and Tatsunori Hashimoto. 2024. Identifying the risks of lm agents with an lm-emulated sandbox. In *International Conference on Learning Representations*, volume 2024, pages 27031–27098.

Manan Suri, Puneet Mathur, Nedim Lipka, Franck Dernoncourt, Ryan A. Rossi, and Dinesh Manocha. 2026. Structured uncertainty guided clarification for LLM agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 40811–40838, San Diego, California, United States. Association for Computational Linguistics.

Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. 2023. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. In *Advances in Neural Information Processing Systems*, volume 36, pages 38975–38987. Curran Associates, Inc.

Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2026. Agentspec: Customizable runtime enforcement for safe and reliable llm agents. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, pages 2938–2950.

Zhiyuan Wang, Qingni Wang, Yue Zhang, Tianlong Chen, Xiaofeng Zhu, Xiaoshuang Shi, and Kaidi Xu. 2025. SConU: Selective conformal uncertainty in large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 19052–19075, Vienna, Austria. Association for Computational Linguistics.

Bosi Wen, Pei Ke, Xiaotao Gu, Lindong Wu, Hao Huang, Jinfeng Zhou, Wenchuang Li, Binxin Hu, Wendy Gao, Jiaxin Xu, Yiming Liu, Jie Tang, Hongning Wang, and Minlie Huang. 2024. Benchmarking complex instruction-following with multiple constraints composition. In *Advances in Neural Information Processing Systems*, volume 37, pages 137610–137645. Curran Associates, Inc.

Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. 2025. GuardAgent: Safeguard LLM agents via knowledge-enabled reasoning. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 68316–68342. PMLR.

Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. 2024. TravelPlanner: A benchmark for real-world planning

with language agents. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 54590–54613. PMLR.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems*, volume 35, pages 20744–20757. Curran Associates, Inc.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Michael JQ Zhang and Eunsol Choi. 2025. Clarify when necessary: Resolving ambiguity through interaction with LMs. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 5541–5558, Albuquerque, New Mexico. Association for Computational Linguistics.

Zihan Zhu, Shayan Kiyani, George Pappas, and Hamed Hassani. 2026. Conformal risk-averse decision making with action conditional guarantee. arXiv preprint arXiv:2606.05551.

Appendix guide. A: semantics and interface; B: tasks and protocol; C: complete primary results; D: retained-intent decisions; E: execution controls, robustness, and cost; F: targeted diagnostics.

A Formalization and Executable Interface

A.1 Decision semantics and assumptions

RAC supplies the fixed-set risk-averse decision principle (Kiyani et al., 2025). Our decision analysis fixes $x = (P, c)$, a nonempty retained state set $\Theta(x)$, and state-dependent hard requirements, risk permissions, and utilities before optimization. Utilities share a cardinal scale and are bounded over the finite experimental domain. The representation is assumed to preserve decision-relevant interpretations and their associated conditions and consequences. This does not require a uniquely resolved intent, a distribution over states, linearity, or convexity. No conformal coverage or action-conditional calibration is implemented.

One action across readings. For "Buy tickets for Wang and Li; his ticket must be refundable," retain θ_W and θ_L. One candidate z specifies both tickets. Membership in $F_{\theta_W}(x) \cap F_{\theta_L}(x)$ requires both refundability conditions and the shared budget. Buying two refundable tickets is compatible with each reading; it does not assert that the user explicitly requested both to be refundable. A common positive-value purchase can be selected without resolving the referent or asking the user. Controlled evaluations of this behavior are in Appendix D.

A.2 Representation and evaluation

Each state record contains a label and expressions `hard`, `risk`, and `utility`. Boolean expressions support conjunction, disjunction, negation, comparisons, and conditionals. Arithmetic supports exact literals, basic operations, minimum, maximum, and absolute value. Boolean and numeric values have distinct types. The attribute schema grounds expression variables; an unavailable applicable attribute cannot certify feasibility. The primary benchmark provides complete attributes.

For generated states $j = 1, \ldots, m$, evaluation gives

$$\widehat{f}(z) = \bigwedge_{j=1}^{m} (\widehat{H}_j(z) \wedge \widehat{R}_j(z)), \tag{5}$$

$$\widehat{\psi}(z) = \min_{1 \le j \le m} \widehat{u}_j(z). \tag{6}$$

The fixed procedure first requires complete enumeration of C, a decided $\widehat{f}(z)$ for every candidate, and known state utilities whenever $\widehat{f}(z) = 1$. An aggregate predicate with a known false conjunct is false even if another conjunct is unknown. Failure to establish these conditions produces FAIL(uncertified), before any purchase is selected. The procedure then forms $\widehat{F}_C = \{z \in C : \widehat{f}(z) = 1\}$ and maximizes $\widehat{\psi}$ over that domain. Any nonempty subset of tied positive-value maximizers is valid. An empty domain or nonpositive maximum produces refusal. Invalid expressions produce an interface/execution failure, not a certified refusal. State labels alone do not affect the decision; equivalent merged expressions may encode the same functions.

Exactness on the declared domain. After these checks, membership in $\widehat{F}_C$ is decided for every $z \in C$, and all values over that domain are known. Finite enumeration therefore returns the complete set of maximizers when the maximum is positive. Otherwise, an empty domain or a nonpositive maximum proves that the compiled problem has no positive-value option. No unresolved candidate is silently excluded from the optimization.

For unresolved inputs, the historical experiment executor operates only on the certified observed subset. Such outputs fall outside Algorithm 1's exact-solution contract. Recorded outcome accuracy compares returned actions and values with the reference; a matching no-action outcome does not by itself certify infeasibility. Runtime reasons distinguish insufficient evidence from an established negative result.

These hatted functions are instantiated by the model. The reference F and ψ enter offline scoring, never runtime selection. Execution is exact for the generated expressions; correctness relative to the request additionally requires faithful state semantics.

Concrete syntax and serialization. The implementation serializes intent-DSL instances as JSON, a carrier for the specification. Each state record contains a label and expressions for hard requirements, risk admissibility, and utility; their meaning comes from the expression language and evaluation rules. An illustrative record is:

```
{"label": "service_failure",
 "hard": "price <= 100000",
 "risk": "max(0,loss-cover) <= 20000",
 "utility":
   "110000-price-max(0,loss-cover)"}
```

Here the schema defines price, gross loss, and compensation in integer monetary subunits. This record encodes the failure-state conditions in Table 3; a full specification includes the nonempty state list and runtime metadata. It is an interface illustration, not an observed model output. State-indexed semantics, rather than JSON keys alone, define the intent DSL.

Carrier decoding and bounds. A shared schema-aware decoder may recover a uniquely identifiable schema-valid JSON object without consulting reference answers; it does not edit expression meaning. The expression engine permits 200,000 characters per expression, 100,000 AST nodes, depth 100, 256 states, 10,000 candidates, and one million state–candidate combinations. All primary problems are expressible within these bounds. The Python control allows loops, functions, conditionals, `math`, `itertools`, and exact fractions, with four CPU seconds/eight wall seconds per catalogue and a one-MiB output bound. A reference program, withheld from models, verifies that the largest primary catalogue fits this execution budget.

A.3 Output contracts and finite realization

For completeness, distinguish a compiled model Q, whose pre-selection domain is $\widehat{F}_C$, from an output contract Q_{out}, whose denotation $S_x(Q_{\text{out}}) \subseteq \mathcal{Z}$ contains its permitted purchases. Relative to reference semantics, an accepted contract satisfies

$$\varnothing \neq S_x(Q_{\text{out}}) \subseteq F(x). \tag{7}$$

Its guaranteed value is

$$W_x(Q_{\text{out}}) = \inf_{z \in S_x(Q_{\text{out}})} \psi_x(z). \tag{8}$$

For a promised value $t > 0$, feasibility and $W_x \geq t$ are equivalent to

$$\begin{aligned} &\varnothing \neq S_x(Q_{\text{out}}) \subseteq K_t(x), \\ &K_t(x) = \{z \in F(x) : \psi_x(z) \geq t\}. \end{aligned} \tag{9}$$

Indeed, membership gives feasibility and the pointwise bound $\psi_x(z) \geq t$; conversely, a lower bound on the infimum bounds every permitted purchase. These are reference obligations, not an extra runtime oracle. Restricting a sound outcome set preserves its requirements; deleting a predicate may instead enlarge the domain.

Offer	Price	Max. loss	Min. utility	Allowed?
A	800	250	50	No
B	850	120	130	Yes
C	900	100	100	Yes

Table 3: **Illustrative decision trace.** All monetary amounts are in MU. B has the highest guaranteed value among risk-authorized offers. This hand-checkable example explains the method; it is not an experimental observation.

Proposition 1 (Finite contract realization)**.** *For finite C, exact evaluation, bounded utility, and a language denoting every singleton in C, if $F_C = F(x) \cap C \neq \varnothing$, the best guaranteed value of a nonempty admissible contract in C equals* $\max_{z \in F_C} \psi_x(z)$.

Proof. A maximizer z^* exists on finite F_C. Every nonempty $S \subseteq F_C$ has $\inf_{z \in S} \psi_x(z) \leq \psi_x(z^*)$; a contract denoting $\{z^*\}$ attains equality. Acceptance improves on the zero outside option exactly when this maximum is positive. If F_C is empty, no nonempty admissible contract exists. □

This conditional expressibility result neither proves that generated predicates are faithful nor gives a statistical guarantee. Context fixes authorization; infeasibility cannot create permission for extra spending. Reuse requires unchanged attribute meanings, requirements, state semantics, and authorization. That property is shared by executable realizations, including Python.

A.4 Illustrative execution trace

A 1,000-MU budget, a 200-MU net-loss cap, and utility $1{,}100 - \text{price} - \text{net loss}$ give Table 3. Normal delivery has zero loss; the listed failure determines the worst case. A is unauthorized despite positive value. B and C pass, and B has higher guaranteed value. A catalogue containing only A is refused; one containing only C is accepted. These stipulated numbers illustrate the procedure, not a measured model output.

B Benchmark Construction and Experimental Protocol

B.1 Contract structures and paired boundaries

The primary dataset contains 20 templates (Table 4), with five feasible/infeasible pairs per template. Each purchase has six components; the retained events are the 57 affected subsets of size

at most four, including the empty event. Empty events have zero assessment. A nonempty event incurs one administrative fee, outside insurance or compensation offsets. Public contracts state that no additional hidden clauses apply.

The master construction seed is 39071007, with predetermined template and stratum offsets. In each feasible instance the unique authorized positive-value candidate has the margin in Table 5. Increasing only its administrative fee makes the paired task infeasible: its maximum loss becomes 200,001 subunits, against a 200,000-subunit limit. Other candidates already exceed the limit by 1–20,000 subunits. Hashed candidate identifiers carry no answer labels; candidate positions are sampled before attributes are generated. Monetary reporting uses 100 integer subunits per MU without changing frozen prompts or calculations.

Unique feasibility, balanced acceptance/refusal, and complete catalogues are controlled design choices. The strata change size, margin, and seed together; they do not identify separate causal effects of those factors or estimate real purchasing frequencies.

B.2 Reference calculations and freezing

A cash-flow ledger evaluates bitmask events with integer or rational arithmetic; a separate expression implementation uses the rule engine. Before model evaluation, they agree on each candidate's feasibility, worst loss, guaranteed utility, and the final decision: 6,712,320 candidate–event checks. Twenty fresh attribute assignments per template are also checked on all 64 subsets, yielding $20 \times 20 \times 64 = 25{,}600$ checks, plus three hand-calculated threshold witnesses. The implementations arose within one development workflow; agreement does not exclude a shared misunderstanding of a clause.

Tasks, references, prompts, and evaluator code are frozen before model evaluation. Public inputs exclude construction seeds, reference matrices, labels, and designated safe identifiers. Candidate visibility follows Table 6.

B.3 Common instructions and outcome protocol

All primary routes receive the same substantive policy, summarized below. Frozen model-facing prompts retain their original language; this English rendering describes the policy rather than replacing those prompts.

> Retain applicable interpretations. Respect current authorization in every retained state. Evaluate hard requirements and the complete risk permission, then maximize the minimum state-dependent utility. Buy only for positive guaranteed value; otherwise refuse. Do not ask the user or change permissions.

Direct returns a decision, allowed identifiers, and guaranteed utility. Set returns the state-wise expressions of Appendix A.2. The supplementary procedural control is defined in Appendix E.2. Exactness requires the reference decision, authorized maximizing purchases, and correct value. Disjoint failures are unsafe acceptance, false refusal, and Other (remaining value or execution failures). Execution failure is never credited as correct refusal.

One logical trial retains one semantic attempt. Identifiable transport failures permit bounded retries; formatting, semantic, and output-budget failures terminate the trial. Every received result is durably saved before scoring. Within a completed cohort all failures remain in its denominator; unissued calls remain missing. Historical results are reused only when messages, schema, model, effort, deadline, budget, and public/private hashes match, with source priority independent of correctness. Other panels state their departures from this protocol locally.

B.4 Model configurations

Table 7 gives the primary and retained-intent settings, followed by a consolidated configuration inventory. The inventory covers 52 observed model–setting combinations across 12 requested model identities. Beyond the main comparison, these settings occur in effort and language studies, ambiguity and risk-parameter tests, catalogue and context controls, budget recovery, rule reuse, and semantic diagnostics. Historical exploratory settings are included for completeness of configuration coverage, without adding them to the primary result denominators. The same settings apply to Direct and Set within a paired comparison. Primary CLI runs use fresh contexts with external tools and delegation disabled. API routes use their native structured-output carrier followed by the common local validator. Requested effort and output ceilings do not imply matched compute across providers.

B.5 Comparison cohorts

Table 8 identifies the sample unit and comparison scope of each study.

ID	Family	Distinct contractual structure
a1	Coupon cancellation	Overlapping three-person groups; a clawback applies to partial but not full group cancellation.
a2	Coupon cancellation	Different settlement charges for one versus both members of a two-person group.
a3	Coupon cancellation	A cap on group clawbacks plus a conditional order-level clawback.
a4	Coupon cancellation	A rebate offsets group charges only after multiple groups are broken.
a5	Coupon cancellation	Only the largest group clawback is charged, with a specified combination surcharge.
b1	Insured cancellation	Item deductibles and item payout limits, followed by an aggregate payout cap.
b2	Insured cancellation	An order-level deductible followed by proportional coinsurance.
b3	Insured cancellation	The number of cancellations changes the aggregate insurance limit.
b4	Insured cancellation	Primary insurance followed by secondary insurance on remaining loss.
b5	Insured cancellation	Payout is triggered by a strict threshold, then calculated from the first unit of loss.
c1	Threshold returns	Two product groups have separate retained-value discount clawbacks.
c2	Threshold returns	Fixed shipping parcels; return freight is the largest item fee within each affected parcel.
c3	Threshold returns	Retained purchase value determines a tiered retained discount.
c4	Threshold returns	Separate retained-value and retained-weight thresholds govern two subsidies.
c5	Threshold returns	Restocking fees are capped; sufficiently large returns waive return freight.
d1	Service replacement	Item-level compensation limits apply before an order-level cap.
d2	Service replacement	Activating replacement service incurs a fixed fee per affected group.
d3	Service replacement	The number of failures determines the aggregate compensation cap.
d4	Service replacement	Cash compensation combines with explicitly authorized, discounted service credits.
d5	Service replacement	Overlapping group dispatch fees and threshold-triggered additional compensation.

Table 4: All 20 primary structures. Numerical instances vary within each template; the four families contain 50 cases each. These contracts define synthetic assessment rules and are not claims about actual merchants' terms.

Stratum	Candidates	Margin (subunits)	Cases
1	128	1	40
2	256	100	40
3	512	10,000	40
4	1,024	100	40
5	1,024	1	40

Table 5: Each stratum includes one pair for every template. Catalogue size, margin, and seed change together; these strata do not isolate their individual causal effects.

Property	Direct	Set
Natural-language records	Yes	Yes
Attribute schema	Yes	Yes
Catalogue in model input	Yes	No
Deterministic row evaluation	No	Yes
Exact arithmetic in executor	—	Yes
Observable allowed domain	No	Yes
Model calls per primary trial	1	1

Table 6: Primary method controls. Different catalogue visibility is a declared architectural choice. "One call" refers to a logical trial; transport retries are separately retained.

B.6 Unified reproduction entry point

The anonymous artifact package provides the implementation, benchmark inputs and references, prompts, model configurations, evaluation records, and frozen reporting inputs. It includes 800 original request/response records covering four primary DeepSeek Direct/Set cohorts, 8,046 archived parsed-output records, and 3,520 supporting records for the prompt engineering and execution controls (Appendix E.1), including 1,000 new checklist calls. These overlapping collections serve distinct inspection and reporting purposes; they are not independent task counts. Reporting inputs distinguish per-trial observations from aggregate updates. The package is available at `https://anonymous.4open.science/r/risk-aware-payment-delegation-D4FC/`. After installing the declared dependencies, run from the repository root:

```
python reproduce.py --pdf
```

This offline entry regenerates result tables and the four main figures from frozen reporting inputs, checks Figure 3's paired outcomes and error partitions, runs deterministic checks of Algorithm 1, verifies retained raw-record hashes and their alignment with published outcome rows, and compiles the manuscript. Omitting `--pdf` reproduces the analysis without TeX. Adding `--verify-references`

Configuration	Requested model	Output cap	Deadline
Primary 200-task comparison			
DeepSeek low / high	`deepseek-flash`	262,144	1,800
Qwen Flash low	`qwen3.8-flash`	131,072	1,800
Qwen Max low	`qwen3.8-max`	131,072	1,800
GLM Flash low	`glm-5.3-flash`	131,072	1,800
GLM high	`glm-5.3`	131,072	1,800
5.6 Luna medium	`gpt-5.6-luna`	128,000	1,800
5.6 Terra medium	`gpt-5.6-terra`	128,000	1,800
5.6 Sol medium	`gpt-5.6-sol`	128,000	1,800
Astra max	`gpt-6-astra`	128,000	1,800
Opus 5.5 medium	`claude-opus-5-5`	128,000	1,800
Retained-intent comparison (Figure 3)			
DeepSeek low / max	`deepseek-flash`	32,768	300
GLM low	`glm-5.3`	32,768	300
Qwen Max low	`qwen3.8-max-0902`	32,768	300

Table 7: Requested model configurations. Output caps are in tokens and deadlines in seconds. Each named primary configuration uses 200 tasks per method. The lower block gives the four configurations in Figure 3; their task counts appear in Table 12. Configuration names refer to requests, not independently attested backend versions.

Configuration	Requested model	Output caps	Deadlines
DeepSeek Flash off, low, high, max	`deepseek-flash`	8,192, 16,384, 24,000, 32,768, 48,000, 65,536, 262,144	180, 240, 300, 360, 420, 600, 900, 1,800
DeepSeek V4 Pro off, low, high, max	`deepseek-v4-pro`	32,768	300
Qwen Flash off, low, medium, xhigh	`qwen3.8-flash`	24,000, 32,768, 131,072	300, 420, 1,800
Qwen Max low, xhigh	`qwen3.8-max`	131,072	1,800
Qwen Max (0902) off, low, medium, xhigh	`qwen3.8-max-0902`	32,768	300
GLM Flash low, high, max	`glm-5.3-flash`	24,000, 32,768, 65,536, 131,072	300, 420, 1,800
GLM low, high, max	`glm-5.3`	32,768, 65,536, 131,072	300, 1,800
5.6 Luna off, low, medium, high, xhigh, max	`gpt-5.6-luna`	128,000	1,800
5.6 Terra off, low, medium, high, xhigh, max, ultra	`gpt-5.6-terra`	128,000	1,800
5.6 Sol off, low, medium, high, xhigh, max, ultra	`gpt-5.6-sol`	128,000	1,800
Astra low, medium, high, xhigh, max, ultra	`gpt-6-astra`	128,000	300, 600, 900, 1,800
Opus 5.5 medium, max	`claude-opus-5-5`	128,000	900, 1,800

Table 7: Requested model configurations (continued): complete recorded model–setting coverage. Off denotes the native none setting. Output caps are in tokens and deadlines in seconds. Multiple values summarize settings observed in separate studies; they are not a fully crossed experimental grid. The primary and retained-intent settings are specified in the preceding part. Performance comparisons retain their own study cohorts.

Comparison cohort	Sample unit	Analysis
Primary decisions	200 tasks per method and configuration	Exact decisions, unsafe acceptance, false refusal, and feasible-task completion.
Execution controls	Separate matched implementation runs	Attribution of executable decision gains under a shared outer decision policy.
Task language	Matched Chinese/English tasks; 60-task and 200-task panels reported separately	Method and language effects within the same task panel.
Semantic diagnostics	Frozen rules and controlled interventions	Admitted-domain errors and the effects of changing candidate availability or state semantics.

Table 8: Comparison cohorts. Statistics are computed within their designated task sets; separate runs and panels are not pooled into a single success rate.

recomputes the 200 primary reference decisions with the exact interpreter and separate cash-flow ledger. The command makes no model calls and does not modify experimental records. Its reports record the checks, input hashes, and regenerated outputs; Table 7 specifies generation settings. Reconstructing tables from reporting inputs is distinct from re-scoring raw responses.

C Complete Results and Statistical Analysis

C.1 Primary outcomes and matched comparisons

Table 9 consolidates the comparison in Figure 1: eleven configurations with 200 tasks per method. Error categories and feasible-task completion share the same cohort definitions. Cohort definitions are in Table 8.

C.2 Structural coverage

Table 10 decomposes the primary DeepSeek comparison over all 20 ten-task templates. Each setting improves over Direct on 18 templates, ties on one, and declines on one. The four families have 50 cases each; low-effort service replacement changes from 20 to 42 exact decisions, and high-effort threshold returns from 30 to 49. Astra's 180-to-196 aggregate is reported in Table 9.

C.3 Paired statistical analysis

For a configuration and metric, compute a mean paired difference d_k over the cases of template k. For the pooled DeepSeek comparison, both configurations remain inside the same template cluster. Resample 20 template indices with replacement, average their d_k, repeat 20,000 times with seed 20261008, and take the 2.5th/97.5th percentiles. This preserves feasibility-pair and template dependence (Dror et al., 2018); 20 clusters limit precision. Table 11 reports descriptive percentage-point intervals, without multiplicity-adjusted hypothesis claims. The 400 pooled outcomes per method represent 200 tasks under two configurations, not 400 independent problems.

Execution controls. Appendix E consolidates the essential procedural controls and corresponding uncertainty intervals. Appendix F separately examines admitted-domain errors beyond the selected outcome.

D Controlled Decisions under Retained Intent Ambiguity

D.1 Task semantics and controlled inputs

The frozen parameter-boundary campaign contains three families with unresolved user interpretations. We include all three, selecting by task semantics; five other families with conditional but unambiguous requirements are excluded. This retrospective study is separate from the contractual-event primary benchmark.

Ticket-holder reference. Two male travellers need tickets; "his" ticket must be refundable with a fee below the authorized limit. Utility penalizes that same traveller's delay and refund fee. Both referents remain possible, linking each interpretation's hard requirement, risk bound, and value function.

Hotel reference and conditional binding. Two guests need correctly registered rooms. "His room" must permit cancellation at 18:00 if he arrives at or after 18:00, or 24 hours before his expected arrival otherwise. The cancellation fee and utility weights refer to the same guest, so the unresolved antecedent affects conditional requirements and value.

Distributive versus collective scope. Gifts must satisfy recipients' preferences and wrapping re-

Model	n	Exact ↑	Unsafe ↓	Refusal ↓	Other ↓	Accept / feasible ↑
DeepSeek low	200	117 → **185**	48 → **3**	24 → **1**	11 → **11**	45/100 → **94/100**
DeepSeek high	200	131 → **185**	31 → **6**	29 → **0**	9 → **9**	55/100 → **92/100**
Qwen Flash low	200	66 → **194**	52 → **1**	26 → **0**	56 → **5**	20/100 → **99/100**
Qwen Max low	200	59 → **87**	49 → **46**	57 → **41**	35 → **26**	0/100 → **26/100**
GLM Flash low	200	29 → **137**	95 → **31**	10 → **6**	66 → **26**	17/100 → **68/100**
GLM high	200	64 → **188**	20 → **5**	4 → **0**	112 → **7**	34/100 → **94/100**
5.6 Luna medium	200	41 → **91**	129 → **75**	28 → **7**	2 → **27**	13/100 → **39/100**
5.6 Terra medium	200	81 → **158**	78 → **24**	40 → **3**	1 → **15**	24/100 → **75/100**
5.6 Sol medium	200	137 → **196**	21 → **2**	42 → **2**	0 → **0**	49/100 → **97/100**
Astra max	200	180 → **196**	6 → **2**	6 → **2**	8 → **0**	87/100 → **97/100**
Opus 5.5 medium	200	142 → **194**	8 → **1**	14 → **1**	36 → **4**	76/100 → **98/100**

Table 9: **Primary comparison: Direct→Set.** Counts use the stated n. Exact requires decision and value correctness. Unsafe, Refusal, and Other partition failures. Accept is exact completion among feasible cases. Execution controls appear in Table 16.

Template	DS low	DS high
a1	6 → 10	8 → 9
a2	8 → 10	6 → 10
a3	6 → 10	7 → 10
a4	5 → 10	7 → 10
a5	8 → 8	7 → 10
b1	7 → 10	9 → 10
b2	8 → 10	7 → 9
b3	7 → 9	9 → 10
b4	6 → 10	7 → 7
b5	7 → 9	5 → 9
c1	6 → 10	5 → 10
c2	4 → 9	7 → 10
c3	4 → 9	7 → 9
c4	7 → 10	6 → 10
c5	8 → 9	5 → 10
d1	3 → 9	4 → 9
d2	3 → 8	7 → 8
d3	9 → 6	8 → 7
d4	5 → 10	6 → 10
d5	0 → 9	4 → 8

Table 10: Primary template coverage: Direct→Set exact decisions, each out of ten. All 20 templates are included; family boundaries follow Table 4.

Control	Metric	Difference	95% interval
Direct	Exact	+30.50	[+22.75, +37.75]
Direct	Unsafe	-17.50	[-24.75, -10.25]

Table 11: Paired template-bootstrap differences, SET minus the listed control, on the pooled DeepSeek primary results. Larger exactness and smaller unsafe rate are desirable. Intervals are descriptive, without multiplicity-adjusted hypothesis claims.

quirements. "Buy one for each; the exchange top-up is at most r" may bound each recipient or the batch. Stipulated utility uses the corresponding maximum individual or total top-up. With nonnegative charges the batch bound may dominate; the correct bound and score still need to be retained.

All requests explicitly signal unresolved reference or scope. Methods see identical user text, complete catalogue attributes, synthetic utility formulas, and the common decision policy. These historical compilation calls see the catalogue, unlike the primary Set protocol. No reference expressions, answer, or subsequent user clarification is supplied. The study tests decisions with retained meanings, not automatic discovery of unrestricted ambiguity.

D.2 Parameter design and findings

Three families, seven settings, and two frozen seeds give 42 API variants. Within each parameter group the candidates and identifiers stay fixed. The authorized risk limit varies over 0, 500, and 1,000 subunits; the risk penalty over 1, 3, and 10; a value offset places the best common candidate at exactly zero or one. Each seed has 18 acceptance and three zero-value refusal variants. These refusals test the value boundary, not an empty feasible intersection. The second seed changes the instance within the same templates.

Figure 3 summarizes 294 Direct/Set outcomes on matched variants: DeepSeek low/max and GLM low each use 42 variants per method, and Qwen Max low uses 21 variants per method. Table 12 gives the four complete outcome partitions. Statewise execution improves exact decisions in each of these four configurations, covering unresolved reference, conditional binding, and collective scope under risk and value constraints. Exactness and safety remain distinct: Qwen has six unsafe Set acceptances versus five for Direct. Reference-arithmetic checks and response replay reproduce the recorded evaluations.

Configuration	n	Exact ↑	Unsafe ↓	False refusal ↓	Other ↓
DeepSeek low	42	40→42	1→0	0→0	1→0
DeepSeek max	42	40→42	0→0	0→0	2→0
GLM low	42	32→36	2→0	0→0	8→6
Qwen Max low	21	1→13	5→6	7→1	8→1

Table 12: Complete results underlying Figure 3. Each arrow is Direct→Set; n is the number of matched tasks per method. Exact, unsafe acceptance, false refusal, and Other form a disjoint partition of each method's n outcomes. Other includes remaining value and execution failures. The four rows contain 294 evaluations in total and are separate from the primary 200-task benchmark.

D.3 Common purchase, empty intersection, and value boundaries

Table 13 adds five earlier base checks. Each is repeated under three context conditions with two samples per method. Direct, rule-reinforced Direct, Merged, and Set each obtain 30/30; reference-assisted Direct has a separate 10/10. These ceiling results verify tested behaviors, not an accuracy advantage. In particular, F02a tests the empty intersection missing from the parameter panel.

The separate input and omitted-state interventions are summarized in Appendix F.2.

E Essential Controls, Robustness, and Reuse

E.1 Prompt Engineering and Executable Decisions

We test whether additional textual reasoning and verification guidance can substitute for executable decision processing. Our *prompt engineering baseline*, abbreviated *Prompt*, appends a generic checklist to the original Direct instruction: identify requirements, check all applicable conditions, verify financial arithmetic and boundaries, compare eligible purchases, and review the final decision. Direct already specifies joint feasibility, worst-case utility, and justified refusal. Thus Prompt tests additional guidance within the same decision policy; neither Direct nor the executable framework is prompt-free.

Controlled inputs. We complete 200 new calls for each of DeepSeek low/high and Sol/Luna/Terra medium. For each Direct–Prompt pair, the recorded task, catalogue, answer schema, model, requested reasoning setting, output-budget configuration, and deadline are unchanged. The appended checklist contains no task-specific examples or reference answers. Both routes return a decision without external execution or further user interaction. DeepSeek uses the original checklist; the native-model version retains its five checks but removes the additional prohibition on internal program-like representations. The comparisons use retained Direct runs, so service-version drift across run dates is not controlled. The two checklist versions are documented with the released prompts, rather than treated as a separate version ablation.

Effect of textual guidance. Table 14 shows lower exactness in four configurations and a one-percentage-point increase for Terra. The observed changes range from -6.5 to $+1.0$ percentage points. This generic prompt engineering intervention does not yield a consistent improvement; the result concerns the tested checklist, not the effectiveness of prompt engineering in general.

Four-method comparison. Table 15 compares Direct, Prompt, Framework + Set, and Framework + Python on identical tasks within each row. The latter two generate executable representations under the same outer admissibility, worst-value, and selection policy. Set compiles without the catalogue, whereas the other three routes receive it; this comparison evaluates complete realizations rather than isolating syntax alone. DeepSeek has all 200 tasks for every method. For Sol, Luna, and Terra, the earlier Python control shares 40 primary tasks: one feasible/infeasible pair per template, selected for that earlier panel independently of outcomes. We verify that their task attributes and reference decisions agree with the primary versions. These paired controls use retained per-trial runs, separately from later aggregate primary updates.

Both executable realizations exceed Direct and Prompt exactness in every row of this matched panel. This supports the value of executable decision processing beyond the tested textual guidance. Differences between Set and Python further show that this benefit is shared across executable realizations rather than exclusive to one encoding.

Base	Retained meanings and candidate control	Reference decision
F01a	Zhang or Li needs a refundable ticket; a 2,800-MU order refunds both within a 3,000-MU budget.	Buy the common refundable order.
F02a	Only two 2,400-MU orders remain: one refunds Zhang, the other Li. Each meaning is separately feasible.	Refuse: the common feasible set is empty.
F02b	Add the 2,800-MU order refunding both to F02a.	Buy: ambiguity itself does not require refusal.
F04a	"His experience" may mean Zhang or Li; two feasible options have utilities $(3, -1)$ and $(0.2, 0.2)$.	Choose the latter by worst-case value.
F04b	Replace $(0.2, 0.2)$ by $(0, 0)$, keeping the other option unchanged.	Refuse: the best guarantee is zero.

Table 13: Recorded capability checks, not new examples substituted for measured tasks. Monetary labels use the paper's MU convention; numerical values are unchanged. Six non-oracle generations per base/method are repeated measurements.

Configuration	n	Direct	Prompt	Prompt U/R/O	Δ (pp)
DS low	200	58.5	53.5	48/27/18	-5.0
DS high	200	65.5	59.0	43/27/12	-6.5
Sol medium	200	68.5	67.5	23/41/1	-1.0
Luna medium	200	20.5	19.0	129/30/3	-1.5
Terra medium	200	40.5	41.5	74/41/2	$+1.0$

Table 14: **Additional prompt engineering on the primary tasks.** Direct and Prompt are exact-decision percentages on the same 200 tasks per configuration; Δ is Prompt minus Direct in percentage points. Prompt U/R/O reports unsafe acceptance, false refusal, and other errors. Every new call remains in the denominator.

Configuration	n	Direct	Prompt	Framework + Set	Framework + Python
DS low	200	58.5 48/24/11	53.5 48/27/18	92.5 3/1/11	94.5 6/0/5
DS high	200	65.5 31/29/9	59.0 43/27/12	92.5 6/0/9	93.5 12/0/1
Sol medium	40	75.0 3/7/0	67.5 4/9/0	100.0 0/0/0	100.0 0/0/0
Luna medium	40	25.0 23/7/0	12.5 26/8/1	45.0 16/0/6	100.0 0/0/0
Terra medium	40	40.0 14/10/0	42.5 12/11/0	80.0 6/0/2	97.5 0/0/1

Table 15: **Textual guidance and two executable realizations.** Each cell gives exact-decision percentage above unsafe/false-refusal/other counts. All four methods use the same n tasks within a configuration. The 40-task rows cover all 20 templates and use the pre-existing task intersection. The panel contains 2,080 method outcomes on overlapping cohorts; it is not an additional set of 2,080 tasks.

E.2 Execution Controls: Risk and Useful Completion

The procedural control generates a Python `assess(attrs)` function under the same outer admissibility, worst-value, and selection policy as Set. It is a program-aided control, not an exact reproduction of PAL or Logic.py. Primary and effort controls receive the catalogue; the matched-input control and Set compile without it. Table 16 reports risk, useful completion, and exactness for the primary DeepSeek comparison, all completed effort groups, and the identical-input control.

In the two primary DeepSeek configurations, both realizations complete 186/200 feasible outcomes, while State-wise records nine unsafe acceptances versus eighteen for the procedural control across 400 outcomes per method. The unsafe-rate difference is -2.25 percentage points (95% template-bootstrap interval $[-6.75, +0.75]$). This is a cohort-specific observation: procedural exactness matches or exceeds State-wise in several other groups. The control therefore separates risk, completion, and exactness rather than treating any one metric as sufficient. Systematic representation optimization is left to separate work.

E.3 Requested reasoning settings

The fixed 60-case panel combines ten historical feasible/infeasible pairs with one pair from each of the 20 primary templates. Table 17 details 18 GPT-5.6 configurations from reasoning off through max. The full effort study covers 25 model–effort configurations and three methods, yielding $25 \times 3 \times 60 = 4{,}500$ outcomes; its grouped execution controls appear in Table 16.

Set improves over Direct in 17 of the 18 listed configurations, with Luna off declining from 25 to 14. The panel overlaps earlier and primary tasks, so it is a configuration check rather than independent task-distribution generalization.

E.4 Task language

The original study uses the same 60 identifiers for four profiles, two languages, and three methods: 1,440 outcomes. A reviewed 183-phrase map preserves identifiers, quantities, types, events, and authorization; system-level method instructions stay fixed. Table 18 gives Direct/Set results on the 60-task panel and a separate 200-task Astra panel. The two panels have different task counts and are reported separately.

For DeepSeek low, Direct exactness decreases from 36 to 34 under translation while unsafe acceptances fall from nine to six. Set has zero unsafe acceptances in these cells; their residual errors include invalid outputs. Translation therefore affects safety and exactness differently. This controlled task-language test is neither expert linguistic annotation nor a change of the entire system language.

E.5 Output-budget control

The Direct budget control pairs original output caps with a 262,144-token cap on 46 historically budget-sensitive tasks, yielding 92 new calls. Each pair uses the same 1,800-second deadline and differs only in output cap. Exact decisions increase from 10/46 to 26/46 (Table 19). These new attempts do not overwrite first outcomes.

The larger budget recovers some outputs but leaves unsafe decisions and false refusals. It addresses a resource failure mode without constituting semantic repair.

E.6 Rule reuse and resource accounting

Two fixed requirements each have eight frozen catalogue snapshots. Direct invokes the model per snapshot; Set compiles once per requirement and reuses the same rule. Table 20 includes initial compilation. Summed request durations measure serial request cost, not parallel wall-clock latency. Token totals use recorded input plus output, without assuming equal provider prices.

Astra is exact on all 16 snapshots for both routes; DeepSeek Direct is exact on 15/16 and Set on 16/16. Reusing a rule requires unchanged authorization and state semantics, not merely a previously correct answer.

F Targeted Semantic Diagnostics

F.1 Frozen-rule replay

This diagnostic uses 648 frozen DeepSeek artifacts: 72 tasks, low/high/max settings, and three realizations (Set, matched-input Python, and catalogue-visible Python). The tasks cross six templates, three authorization conditions, two catalogue sizes, and feasible/infeasible pairs. All realizations use the same selector without private reference information. The artifacts are executed on two new catalogues: 1,296 local replays, with no further model calls. Legal candidate attributes are retained, administrative cost is set to zero, and only the first component's replacement/loss attribute changes.

Configuration	n	Unsafe ↓	Safe / feasible ↑	Exact ↑
Primary decisions				
DS low	200	$6 \to 3$	$93/100 \to 94/100$	$189 \to 185$
DS high	200	$12 \to 6$	$93/100 \to 92/100$	$187 \to 185$
Requested effort				
Astra (6 settings)	360	$0 \to 0$	$180/180 \to 180/180$	$360 \to 360$
Sol (7 settings)	420	$0 \to 11$	$190/210 \to 186/210$	$382 \to 383$
Terra (6 settings)	360	$0 \to 26$	$172/180 \to 149/180$	$342 \to 302$
Luna (6 settings)	360	$0 \to 87$	$178/180 \to 92/180$	$358 \to 224$
Matched catalogue input				
DS (3 settings)	216	$7 \to 7$	$102/108 \to 101/108$	$207 \to 201$
Astra (3 settings)	216	$0 \to 0$	$108/108 \to 107/108$	$216 \to 215$

Table 16: **Risk and useful completion under execution controls.** Arrows are Procedural→State-wise; n is per method. Unsafe counts unauthorized purchases; Safe requires exact purchase and utility on feasible tasks. Effort groups cover all 25 completed configurations. Matched-input controls use low/high/max, with 72 tasks per setting. Cohorts and reused Set results are not pooled across groups.

		Direct		Set	
Model	Setting	Exact/n	U/R/O	Exact/n	U/R/O
Sol	off	0/60	60/0/0	49/60	8/1/2
	low	30/60	9/20/1	58/60	2/0/0
	medium	41/60	3/16/0	60/60	0/0/0
	high	43/60	0/17/0	60/60	0/0/0
	xhigh	46/60	0/14/0	59/60	0/0/1
	max	45/60	0/15/0	60/60	0/0/0
Terra	off	0/60	59/0/1	27/60	8/3/22
	low	24/60	17/19/0	53/60	6/0/1
	medium	26/60	15/19/0	52/60	6/0/2
	high	33/60	7/20/0	52/60	5/0/3
	xhigh	37/60	8/14/1	59/60	1/0/0
	max	33/60	3/15/9	59/60	0/0/1
Luna	off	25/60	6/28/1	14/60	24/12/10
	low	6/60	44/10/0	19/60	28/6/7
	medium	13/60	40/7/0	32/60	20/2/6
	high	24/60	28/8/0	48/60	10/0/2
	xhigh	35/60	11/13/1	53/60	3/0/4
	max	41/60	4/12/3	58/60	2/0/0

Table 17: GPT-5.6 reasoning settings from off through max: exact/n and unsafe/false-refusal/other (U/R/O), with $n = 60$ in every cell. The 18 configurations cover Sol, Terra, and Luna. Settings denote requested effort, not equal or measured compute.

The mixed catalogue targets the following reference worst losses in integer subunits:

> 100000, 199999, 200000, 200001, 210000, 225000, 250000, 300000, 350000, 399999, 400000, 400001, 500000, 600000.

The second catalogue contains only the 400,000-subunit offer, above the 200,000 limit. Other hard constraints remain satisfied. Numerical and symbolic references agree on 53,460 candidate–state checks. These directed perturbations test the generated consequence functions, not a randomly sampled purchasing population.

Valid mixed-catalogue replays make no unsafe selections, while 23 artifacts admit an unsafe candidate. Unsafe-only replays yield eight unsafe selections; each condition has 12 invalid executions. Table 21 gives the DeepSeek counts. The visible-input realization additionally tests transfer from its compilation catalogue; matched-input realizations exclude catalogue attributes during compilation.

F.2 Omitted-interpretation and component interventions

The source pool contains 56 Qwen/GLM Set generations for one ambiguous ticket request and two catalogues (128 and 512 offers). Sixteen outputs admit a reference-unsafe candidate and are selected for diagnosis. Each omits the reading in which "his" refers to Li, retaining only Zhang's fee-free-refund conditions and delay-weighted utility. These are 16

Configuration	Language	Direct Exact/n	Direct U/R/O	Set Exact/n	Set U/R/O
DS low	ZH	36/60	9/14/1	59/60	0/0/1
	EN	34/60	6/15/5	58/60	0/0/2
DS high	ZH	34/60	10/14/2	55/60	0/0/5
	EN	33/60	8/16/3	58/60	0/0/2
DS max	ZH	35/60	11/13/1	57/60	2/0/1
	EN	34/60	12/11/3	58/60	0/1/1
Astra max (60)	ZH	55/60	0/4/1	60/60	0/0/0
	EN	49/60	0/9/2	60/60	0/0/0
Astra max (200)	ZH	180/200	6/6/8	196/200	2/2/0
	EN	180/200	6/6/8	196/200	2/2/0

Table 18: Language outcomes and failure partitions in one table. U/R/O denotes unsafe/false-refusal/other. Panels with $n = 60$ and $n = 200$ are identified separately. Figure 2b shows the DeepSeek subset.

Configuration	Method	n per cap	Exact	Budget	Unsafe	Refusal	Other
DS high	Direct	21	$4 \to 12$	$11 \to 0$	$0 \to 3$	$6 \to 6$	$0 \to 0$
DS low	Direct	25	$6 \to 14$	$11 \to 1$	$0 \to 1$	$7 \to 7$	$1 \to 2$

Table 19: Original→expanded output cap for Direct; the columns partition each row's outcomes. Other denotes a remaining value error. Selection of historically budget-sensitive tasks limits prevalence estimates.

Model	Method	Model calls	Input + output tokens	Serial request seconds
Astra max	Direct	16	175,429	344.64
	Set contract	2	22,160	81.69
DeepSeek low	Direct	16	136,345	351.47
	Set contract	2	12,418	33.61

Table 20: Direct/Set reuse costs including cold compilation, over 16 catalogue evaluations per row. Both Astra routes and DeepSeek Set are exact on all snapshots; DeepSeek Direct is exact on 15/16. Summed request durations are serial cost measures, not parallel wall-clock latency.

faulty outputs, not 16 independent requests. Restoring Li's state restores its conditions and utility; duplicating Zhang's label cannot do so.

The original rules select safe purchases but have incorrect values and admitted domains. Reference admissibility alone repairs the domain; reference utility alone makes the unsafe offer win. Both components, or the correctly restored state, recover exact decisions in all 16 selected outputs (Table 22). These offline interventions diagnose semantic content and are not an inference-time discovery or repair algorithm.

F.3 Input intervention on the same request

Using the 512-offer request, six configurations, three input conditions, and two samples produces 36 calls, with 12 generations per condition. Configurations are Qwen Max off/medium, Qwen Flash low, GLM Flash low, GLM low, and Astra max. The reference-interpretation condition explicitly preserves both possible referents and the corresponding utility binding, without giving a selected offer or reference formula. It is researcher-supplied input, not an additional user response.

Catalogue withholding alone does not remove the errors. Supplied interpretations yield 11/12 exact decisions and one Other failure; no condition selects an unsafe offer on the original catalogue. This local intervention identifies a representation dependency rather than a population-level success rate.

F.4 Two primary decision examples

Reference evaluation checks all 57 events. In one Astra Direct return-cost case, the reported loss of 1,954.33 MU understates the contractual loss of 2,054.33 MU against a 2,000 MU limit; no offer is feasible. In a coupon-cancellation case, Direct rejects all 1,024 offers although one has positive value and loss of 1,999.99 MU. These primary ex-

Configuration	Frozen realization	Unsafe domain Mixed	Unsafe selection Mixed	Unsafe selection Unsafe-only	Invalid Mixed / only
DS low	Python matched	2	0	1	1/1
	Python visible	2	0	0	3/3
	Set	1	0	1	3/3
DS high	Python matched	1	0	1	0/0
	Python visible	5	0	0	0/0
	Set	2	0	0	1/1
DS max	Python matched	4	0	1	0/0
	Python visible	2	0	1	0/0
	Set	4	0	3	4/4

Table 21: DeepSeek frozen-rule replay counts for low, high, and max settings: 72 artifacts per row and two catalogues per artifact. Unsafe domains and choices are counted separately; invalid executions are not credited as safe. The primary DeepSeek plot pools the three realizations within each configuration.

Intervention	Exact	Unsafe	Unsafe set
Original	0	0	16
Duplicate label only	0	0	16
Ref. admissibility	0	0	0
Ref. utility	0	16	16
Both components	16	0	0
Restore missing state	16	0	0

Table 22: All counts are out of 16 selected faulty specifications. Domain error here means admitting a reference-unsafe candidate. Correct selection, correct value, and correct permission set are separate targets.

Input	Exact/12	Valid/12	Overbroad
Original	5	11	6
Catalogue withheld	6	12	5
Reference interpretations	11	11	0

Table 23: One ambiguous request; six configurations and two samples per condition. Unsafe domains are counted among valid executable outputs (11, 12, and 11 respectively). All outcomes remain in the exactness denominators.

amples illustrate unsafe acceptance and false refusal at the same authorization boundary.